

Day 23: 週次レポートを実装しよう

今日のゴール

週ごとのタスク完了数や作業時間を集計して表示する週次レポート機能を実装します。

【スクリーンショット: 週次レポート画面】

なぜこれを作るのか?

チームの生産性を定期的に振り返る機能です。**週次レポートは家計簿のようなもの**。毎週の収支を記録することで、お金の使い方が見えてきます。同様に、週ごとのタスク完了数を記録することで、チームの働き方やペースが見えてきます。

実装ステップ一覧

ステップ	作業内容	所要時間
Step 1	週次レポートページ作成	10分
Step 2	週選択コンポーネント	15分
Step 3	週次データ取得API	15分
Step 4	折れ線グラフで表示	15分

合計時間: 約55分

Step 1: 週次レポートページ作成 (10分)

実装:

```
// filepath: src/app/report/weekly/page.tsx
'use client';

import { Box, Typography } from '@mui/material';

export default function WeeklyReportPage() {
  return (
    <Box sx={{ p: 3 }}>
      <Typography variant="h4">週次レポート</Typography>
    </Box>
  );
}
```

確認ポイント: /report/weeklyにアクセスして「週次レポート」が表示される

Step 2: 週選択コンポーネント (15分)

実装:

```
// filepath: src/app/report/weekly/page.tsx
import { useState } from 'react';
import { Button, ButtonGroup } from '@mui/material';
import ArrowBackIcon from '@mui/icons-material/ArrowBack';
import ArrowForwardIcon from '@mui/icons-material/ArrowForward';

export default function WeeklyReportPage() {
  const [weekOffset, setWeekOffset] = useState(0);

  const getWeekRange = (offset: number) => {
    const today = new Date();
    const startOfWeek = new Date(today);
    startOfWeek.setDate(today.getDate() - today.getDay());

    const endOfWeek = new Date(startOfWeek);
    endOfWeek.setDate(startOfWeek.getDate() + 6);

    return {
      start: startOfWeek,
      end: endOfWeek,
    };
  };

  const weekRange = getWeekRange(weekOffset);

  return (
    <Box sx={{ p: 3 }}>
      <Box sx={{ display: 'flex', justifyContent: 'space-between' }}>
        <Typography variant="h4">週次レポート</Typography>
        <ArrowForwardIcon />
      </Box>
      <ButtonGroup>
        <Button onClick={() => setWeekOffset(weekOffset - 1)}>
          <ArrowBackIcon />
        </Button>
        <Button disabled>
          {weekRange.start.toLocaleDateString('ja-JP')}
        </Button>
      </ButtonGroup>
    </Box>
  );
}
```

```
        {weekRange.end.toLocaleDateString('ja-JP')}  
      </Button>  
      <Button  
        onClick={() => setWeekOffset(weekOffset + 1)}  
        disabled={weekOffset >= 0}  
      >  
        <ArrowForwardIcon />  
      </Button>  
    </ButtonGroup>  
  </Box>  
</Box>  
</>  
};  
}
```

確認ポイント: 週を前後に切り替えられる

Step 3: 週次データ取得API (15分)

実装:

```
// filepath: src/app/report/weekly/page.tsx
import { api } from '@trpc/react';
import { CircularProgress } from '@mui/material';

export default function WeeklyReportPage() {
  const [weekOffset, setWeekOffset] = useState(0);
  const weekRange = getWeekRange(weekOffset);

  const { data: weeklyData, isLoading } = api.report.getWeeklyReport({
    startDate: weekRange.start,
    endDate: weekRange.end,
  });

  if (isLoading) {
    return (
      <Box sx={{ display: 'flex', justifyContent: 'center' }}>
        <CircularProgress />
      </Box>
    );
  }

  return (
    <Box sx={{ p: 3 }}>
      { /* 週選択UI */ }

      <Typography>完了タスク数: {weeklyData?.completedTasks}</Typography>
      <Typography>作業時間: {weeklyData?.totalHours}時間</Typography>
    </Box>
  );
}
```

確認ポイント: 選択した週のデータが表示される

Step 4: 折れ線グラフで表示 (15分)

実装:

```
// filepath: src/app/report/weekly/page.tsx
import {
  LineChart,
  Line,
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip,
  Legend,
  ResponsiveContainer,
} from 'recharts';
import { Paper } from '@mui/material';

export default function WeeklyReportPage() {
  const { data: weeklyData } = api.report.getWeeklyReport(
    startDate: weekRange.start,
    endDate: weekRange.end,
  ));

  return (
    <Box sx={{ p: 3 }}>
      { /* 週選択UI */ }

      <Paper sx={{ p: 3, mt: 3 }}>
        <Typography variant="h6" sx={{ mb: 2 }}>
          日別タスク完了数
        </Typography>
        <ResponsiveContainer width="100%" height={300}>
          <LineChart data={weeklyData?.dailyStats ?? []}>
            <CartesianGrid strokeDasharray="3 3" />
            <XAxis dataKey="date" />
            <YAxis />
            <Tooltip />
            <Legend />
            <Line
              type="monotone"
            />
          </LineChart>
        </ResponsiveContainer>
      </Paper>
    </Box>
  );
}
```

```

        dataKey="completed"
        stroke="#8884d8"
        name="完了タスク"
      />
      <Line
        type="monotone"
        dataKey="created"
        stroke="#82ca9d"
        name="新規タスク"
      />
    </LineChart>
  </ResponsiveContainer>
</Paper>

<Paper sx={{ p: 3, mt: 3 }}>
  <Typography variant="h6" sx={{ mb: 2 }}>
    サマリー
  </Typography>
  <Typography>完了タスク数: {weeklyData?.completedTasks}</Typography>
  <Typography>新規タスク数: {weeklyData?.createdTasks}</Typography>
  <Typography>合計作業時間: {weeklyData?.totalHours}時間</Typography>
</Paper>
</Box>
);
}

```

確認ポイント: 折れ線グラフと統計が表示される

学んだこと

- **Date操作**: `getDay()`, `setDate()` で日付計算
- **週の開始日**: 日曜日を0として計算
- **LineChart**: 折れ線グラフコンポーネント
- **type="monotone"**: 滑らかな曲線
- **ButtonGroup**: ボタンをグループ化して見た目を統一

今日のまとめ

- ☐ 週次レポートページを作成できた
- ☐ 週選択コンポーネントを実装できた
- ☐ 週次データ取得APIを呼び出せた
- ☐ 折れ線グラフで表示できた

⚠ つまづきポイント

問題	原因	解決策
週の計算がずれる	<code>getDay()</code> の仕様を誤解	日曜=0, 月曜=1, ... 土曜=6
未来の週を選択できる	<code>disabled</code> の条件が間違い	<code>weekOffset >= 0</code> で無効化
グラフが表示されない	<code>data</code> が <code>undefined</code>	<code>weeklyData?.dailyStats</code> ?? <code>[]</code> でフォールバック

次回予告

Day 24では、ユーザー一覧画面（管理者用）を実装します。